klzma: LZMA reversibile in Kairos

Dizionario, modelli di contesto e rANS, con la decompressione come `uncall`

Nicolò Giuliani

Università di Bologna, relazione tecnica, ottobre 2026

Sommario

kgzip2 ha mostrato che una pipeline in stile bzip2 si scrive in Kairos una volta sola: `call` comprime, `uncall` decomprime. Questo lavoro fa lo stesso con l'algoritmo di 7-Zip, LZMA: un LZ77 con catene di hash e ripetizione dell'ultima distanza, seguito da un codificatore binario adattivo con i modelli di contesto di LZMA (macchina a 12 stati, letterali ad albero sul byte precedente, codificatore delle lunghezze, slot di distanza). Il range coder di LZMA non si può invertire, perché il suo aggiornamento delle probabilità perde informazione; lo sostituisce un rANS binario, che è una biiezione esatta sugli interi. Il punto che rende pulito un codificatore adattivo reversibile è l'ordine: i gettoni si codificano dall'ultimo al primo, partendo dall'istogramma completo delle decisioni e consumandolo, così che `uncall comprimi` sia esattamente un decodificatore LZMA che parte da un modello vuoto. La parse è golosa con un passo di lazy matching, scelto misurando sul modello Python le varianti di LZMA (lazy, rep1-3, letterale "matched", stimatore delle probabilità). Su file da 8 KB klzma è alla pari con LZMA puro sul testo (3483 contro 3485 byte sulla tesi, CRC compreso), comprime molto meglio di kgzip2 (3816) e fa meglio dello stesso LZMA sui dati ripetitivi (1830 contro 1899 byte sul testo ripetuto). Il ritorno è identico byte per byte su tutti i file, e l'eseguibile `klzma` è un file unico che funziona senza Kairos installato e lavora in streaming, con la memoria costante qualunque sia la dimensione del file.

1 Introduzione

LZMA, il formato di 7-Zip e di xz, combina due idee. La prima è LZ77 [1]: il testo si descrive come una sequenza di letterali e di copie "ripeti ℓ byte da d posizioni indietro". La seconda è una codifica entropica molto fine: ogni decisione (letterale o copia? quale lunghezza? quale distanza?) diventa una serie di bit, e ogni bit si codifica con una probabilità adattiva scelta in base al contesto. Pavlov la realizza con un range coder [2].

kgzip2 ha seguito bzip2, che trasforma il blocco e poi usa un codice fisso. Qui la sfida è diversa: un modello adattivo cambia a ogni bit, e il suo stato finale dipende da tutti i dati. Un inverso che parte dallo stato finale avrebbe bisogno di quello stato, che un file compresso non contiene. I contributi:

1. un **LZ77 reversibile e pulito**: catene di hash, parse golosa con rep0, tutta la ricerca disfatta dopo ogni decisione;
2. la sostituzione del range coder con **rANS binario** [3], e perché è necessaria;
3. l'**ordine di codifica** che rende pulito un codificatore adattivo: istogramma completo, gettoni dall'ultimo al primo, modello che torna a zero;
4. i **contesti di LZMA** codificati in ordine inverso a quello del decodificatore, che l'`uncall` rovescia di nuovo;
5. una **valutazione** contro xz, LZMA puro, bzip2 e kgzip2 sugli stessi file.



decompressione: `uncall comprimi` percorre la catena da destra a sinistra

Figura 1: Il percorso di `klzma` per un gruppo di blocchi, che si ripete finché il file non finisce. In azzurro ciò che esegue Kairos, su ogni blocco del gruppo in un ramo di `par`; le catene di hash servono solo alla parse e spariscono (`uncall catene`) prima dell'istogramma.

2 Perché non il range coder di LZMA

Il range coder di LZMA tiene una probabilità p (su 2048) per ogni contesto e dopo ogni bit la aggiorna con uno scorrimento: $p \mathrel{+}= (2048 - p) \gg 5$ dopo uno zero, $p \mathrel{-}= p \gg 5$ dopo un uno. L'aggiornamento non è iniettivo: per esempio, dopo uno zero, $p = 2016$ e $p = 2017$ finiscono entrambi a 2017. All'indietro non si saprebbe da quale probabilità ripartire. Anche il passo di codifica perde informazione, perché il limite si calcola come $(\mathit{range} \gg 11) \cdot p$ e gli 11 bit bassi dell'intervallo si buttano. Rendere reversibile questo codificatore richiederebbe spazzatura a ogni bit, cioè un'uscita più lunga.

rANS. I sistemi numerici asimmetrici di Duda [3] codificano un simbolo s di frequenza f_s e cumulata c_s (su $M = 4096$) trasformando lo stato intero

$$x' = \lfloor x/f_s \rfloor \cdot M + c_s + (x \bmod f_s).$$

È una biiezione esatta: da x' si legge il simbolo ($x' \bmod M$ cade nell'intervallo di s) e si ricompone $x = f_s\lfloor x'/M \rfloor + (x' \bmod M) - c_s$. Per tenere x limitato, prima di codificare si emettono bit bassi finché $x < 32 f_s$; dopo la codifica $x \in [2^{16}, 2^{17})$. In Kairos un passo è:

```
from x >= 65536 loop                  // si emettono bit finche' x < 32 f
    h += x / 2    x -= h    x -= h    push(x, bitst)    x <=> h
until x < (32 * f)
q += x / f    x -= q * f    x += c    x += q * 4096    q -= x / 4096
v -= (x % 4096) >= p0                  // il bit si azzera decodificandolo
```

L'inverso, generato da `uncall`, è il decodificatore rANS: legge il bit da $x \bmod 4096$, ricompone x, e rilegge bit finché $x \geq 2^{16}$. Il ciclo di rinormalizzazione è un ciclo di Janus valido in entrambi i versi: all'ingresso $x \geq 2^{16}$, dopo un dimezzamento non più.

Probabilità dai conteggi. Al posto dello scorrimento, ogni contesto tiene due conteggi n_0, n_1 e la probabilità dello zero è $p_0 = 1 + \lfloor (n_0+1)(M-2)/(n_0+n_1+2) \rfloor$, sempre in $[1, M-1]$. Contare è iniettivo dato il bit: all'indietro si sottrae. Il prezzo è un modello stazionario dentro il blocco, che non dimentica.

3 `klzma file.txt`, passo per passo

Come per kgzip2 si segue l'eseguibile, che esiste: `dist/bin/klzma`, un binario nativo statico di circa 1 MB con dentro la VM di Kairos, la libreria `klzma.kairos` già tradotta in bytecode e un driver in C. Ha le opzioni di xz: `-z`, `-d`, `-t`, `-l`, `-k`, `-f`, `-c`, i preset `-0`…`-9` (predefinito `-6`), `-e`, `-T`, `-q`, `-v`, `-S`, `--block-size`, con le forme lunghe e i codici d'uscita di xz. Come kgzip2 lavora in streaming, a gruppi di blocchi, con la memoria costante qualunque sia la dimensione del file. Per ogni gruppo il driver scrive il `main` direttamente in bytecode, con i byte e i bit come istruzioni, e lo accoda alla libreria.

3.1 Compressione

1. Blocchi e caricamento. Il file si legge come flusso, a blocchi di dimensione fissa, raccolti in gruppi di K, uno per core (`-T`). Il preset sceglie la dimensione dei blocchi e la profondità della ricerca: da 8 KiB e

4 candidati con `-0` a 256 KiB e 128 candidati con `-9`; il predefinito `-6` vale 64 KiB e 32 candidati, `-e` raddoppia la profondità (Tabella 4). Ogni gruppo è un'esecuzione della VM, con un blocco per ramo di un `par`; in memoria c'è un gruppo alla volta. I byte entrano nello store come istruzioni `PUSHEQ b_0[k] v`, insieme a due tabelle costanti, le potenze di 2 e le basi degli slot di distanza. La libreria è tradotta una volta, quando si costruisce il binario.

2. Catene di hash. Per ogni posizione i si calcola l'hash dei tre byte che partono da lì e si collega i all'occorrenza precedente dello stesso hash: `prev[i] += head[h]; head[h] -= prev[i]; head[h] += i + 1`. Tre assegnamenti reversibili: `head[h]` si azzera sottraendo il vecchio valore, appena copiato in `prev[i]`. Le catene servono solo alla parse, e alla fine si disfano con `uncall catene`.

3. Parse. Da sinistra a destra, in ogni posizione si sceglie un gettone: *letterale*; *match* con lunghezza $\ell \geq 3$ e distanza d; oppure *rep0*, una copia alla stessa distanza dell'ultimo match, che LZMA codifica senza ripetere la distanza. `cerca` percorre al più D candidati della catena (32 per default) e tiene il più lungo; `lung` misura una corrispondenza con l'uscita anticipata già usata per le rotazioni in kgzip2. La regola: rep0 se lunga almeno 2 e non più corta di uno del miglior match, altrimenti match se lungo almeno 3, altrimenti letterale. In più c'è un passo di *lazy matching*, come in zlib: se conviene un match ma in $i+1$ ce n'è uno più lungo, in i si emette un letterale. La seconda ricerca e l'indicatore che ne deriva sono funzioni pure dello stato, e si disfano subito dopo la scelta come il resto. Ogni sostituzione del migliore lascia il vecchio valore su `scarto`, e `decidi` disfa la ricerca subito dopo aver scritto il gettone: la parse è pulita, restano solo i gettoni. Per ciascuno si conservano anche i contesti in vigore prima di esso, lo stato della macchina di LZMA (`st[k]`) e la distanza rep0 (`rp[k]`).

4. Istogramma del modello. Prima di codificare, una passata calcola i conteggi di tutte le decisioni di tutti i gettoni. Le procedure di conteggio (`conta_gettone`, `conta_albero`, ...) seguono lo stesso schema della codifica ma in sola lettura: per ogni bit calcolano contesto e valore dal gettone, senza consumarlo, e fanno `cnt[2 ctx + bit] += 1`. Un assegnamento di questa forma è reversibile da solo, quindi la passata non ha bisogno di nient'altro. Una prima versione rieseguiva la codifica in un "modo conteggio" e poi la disfaceva: costava 9,9 s su 8 KiB (andata e ritorno), questa ne costa 3,4.

5. Codifica all'indietro. I gettoni si codificano dall'ultimo al primo. Per ognuno:

1. si sposta in variabili locali (tipo, lunghezza, distanza, contesti);
2. i suoi byte escono dal blocco: un letterale *è* il byte `b[p]`, che diventa il valore da codificare; un match si toglie dall'ultimo byte al primo, `b[p+j] -= b[p+j-d]`, perché la sorgente è sempre ancora piena;
3. le sue decisioni si codificano in ordine inverso a quello del decodificatore di LZMA: prima il corpo (distanza, lunghezza o letterale), poi `is_rep`, poi `is_match`. Ogni bit decrementa il proprio conteggio e poi passa per il rANS con il modello senza di sé;
4. i contesti `st[k]` e `rp[k]` si azzerano ricalcolandoli dal gettone $k-1$, che è ancora lì.

Gli alberi di bit seguono la stessa logica. Un letterale nel decodificatore si legge dal bit alto, con contesto il prefisso già letto: qui si codifica dal bit basso, e con $v+256$ il dimezzamento lascia il bit in v e il prefisso, cioè il contesto, nella metà. Gli alberi inversi di LZMA (slot bassi e align) si trattano allo stesso modo, specularmente. La distanza segue LZMA: lo slot (albero di 6 bit nel contesto della lunghezza), poi il resto, con albero inverso per gli slot fino a 11 e bit diretti più 4 bit di align oltre.

6. Fine. Blocco, gettoni, contesti e conteggi sono a zero, `scarto` è vuoto, e restano i bit, lo stato rANS finale x e il numero di bit. Il driver scrive subito un frame per blocco, nel formato comune con kgzip2: dopo la firma `KLZ4` e la profondità D, ogni frame tiene il tipo (Kairos o memorizzato, più l'indicatore

pos.	gettone	ℓ	d	stato	rep0	nota
0–4	5 letterali	1	–	0	0	`b a r b a`: `ba` in 3 è lungo 2, troppo poco per un match
5	match	4	1	0	0	quattro `a` copiate dalla precedente: la copia si sovrappone
9	letterale	1	–	7	1	`r`: lo stato dopo un match è 7
10	letterale	1	–	4	1	`a`
11	letterale	1	–	1	1	`!`

Tabella 1: La parse di `barbaaaaara!`: 9 gettoni, 84 decisioni binarie, 70 bit dopo il rANS. I 9 byte dei bit, con lo stato x e il numero di bit, non sono meno dei 12 byte del blocco, che quindi viene memorizzato: su file così corti vale la regola di LZMA2.

dell'ultimo frame), la lunghezza, x e il numero di bit, poi i bit, e un CRC32 del file fino a lì; l'ultimo frame porta anche il CRC32 dei dati originali. Se i bit di un blocco non sono più corti dei suoi byte, il blocco si memorizza tale e quale, come i blocchi non compressi di LZMA2. L'uscita non dipende da K.

3.2 Decompressione: `klzma -d file.txt.klz`

Si procede a gruppi di K frame. Il driver controlla il CRC di ogni frame prima di avviare la VM: un file alterato si ferma al primo frame danneggiato e non arriva all'`uncall`. Il secondo programma carica i bit, x e il numero di bit, ed esegue `uncall comprimi`; i byte ricostruiti si scrivono subito, e dopo l'ultimo frame si controlla il CRC dei dati. L'inverso della codifica all'indietro è una decodifica in avanti, e coincide passo per passo con un decodificatore LZMA:

- l'istogramma iniziale, all'indietro, è il punto d'arrivo: si parte da conteggi a zero e ogni bit decodificato li incrementa, come in qualunque decodificatore adattivo;
- i gettoni si decodificano dal primo, e quando si decodifica il gettone k i precedenti sono già ricostruiti: lo stato, rep0 e il byte precedente, che servono come contesti, sono disponibili;
- ogni gettone si legge nell'ordine di LZMA: `is_match`, poi `is_rep`, poi lunghezza e distanza o il letterale dal bit alto;
- i byte di un match tornano nel blocco con `b[p+j] += b[p+j-d]` dal primo all'ultimo: è la copia LZ77, sovrapposizioni comprese, ottenuta disfacendo la sottrazione della compressione;
- alla fine `uncall analizza` ricalcola la parse dal blocco ricostruito e cancella i gettoni: il blocco resta.

Nessun decodificatore è scritto. Rispetto a kgzip2 qui non serve nemmeno una coppia di procedure: ogni stadio si azzera da sé, per costruzione.

4 L'ordine è la chiave

Un codificatore adattivo reversibile che procedesse in avanti finirebbe con il modello nello stato "dopo tutti i dati", e il suo inverso partirebbe da lì: servirebbe trasmettere tutti i conteggi. Codificando all'indietro a partire dall'istogramma, il modello di partenza dell'inverso è vuoto. In più il rANS è una pila, l'ultimo bit scritto è il primo letto, e la pila si accorda con l'`uncall`, che disfa per primo l'ultimo passo: le due inversioni si compensano, e ne esce un decodificatore nell'ordine naturale. È lo stesso motivo per cui i codificatori rANS ordinari scorrono i dati al contrario; qui lo impone la reversibilità invece dell'efficienza.

5 Valutazione

Ambiente. Lo stesso di kgzip2: Kairos al commit `1aa59fb`, 2 vCPU Xeon, binari statici compilati con `-O3`. Riferimenti: bzip2 1.0.8 `-9`, xz con `-9e`, e LZMA puro (`FORMAT_RAW`, `-9e`), cioè lo stesso

file	byte	bzip2	xz -9e	LZMA	kgzip2	klzma	%	comprimi	`uncall`
tesi (LaTeX)	8192	3343	3544	3485	3816	3483	42,5	5,5	5,8
Janus.c	8192	3214	3172	3113	3601	3124	38,1	4,6	5,0
README	8192	3473	3380	3321	3928	3340	40,8	4,8	5,2
testo ripetuto 2 volte	8192	2182	1956	1899	2934	**1830**	22,3	2,8	3,1
periodico	8192	61	112	54	58	**42**	0,5	1,0	1,1
immagine 64×64	4109	124	168	109	116	123	3,0	0,6	0,7
immagine 16×16	269	78	104	44	63	56	20,8	0,1	0,1
zeri	4096	43	92	32	25	31	0,8	0,5	0,5
casuale	4096	4580	4156	4158	4111	4112	100,4	2,3	0,0

Tabella 2: Byte dopo la compressione con le opzioni predefinite (`-6`: blocchi da 64 KiB, $D = 32$; ogni file è un blocco solo). LZMA è il codificatore di 7-Zip senza intestazione (`xz --format=raw`), xz aggiunge la propria; kgzip2 e klzma includono la propria (firma, tipo e lunghezza del frame) e 8 byte di CRC. In grassetto il più piccolo. Tempi in secondi del comando `klzma`, dall'avvio alla fine; il blocco casuale è memorizzato e non passa per la VM in decompressione. Ritorno identico e uscita uguale al modello in tutte le righe.

codificatore di 7-Zip senza l'intestazione di xz. Opzioni predefinite (`-6`): i file del corpus stanno in un blocco solo.

Correttezza. Ogni uscita di Kairos è confrontata con un modello Python indipendente (`modello.py`), che contiene anche un decodificatore scritto a parte. La suite di test della repository (495 controlli, nessuno fallito, in comune con kgzip2) copre i file della Tabella 2 e 13 casi limite (file vuoto, di 1 e 2 byte, corse lunghe, tutti i 256 byte, due blocchi in `par`): l'uscita compressa coincide byte per byte con il modello, il ritorno con l'originale, e i file alterati o troncati vengono rifiutati dal CRC prima della VM. Per lo streaming prova un file a più frame compresso con gruppi diversi (stessi byte), decompresso con gruppi diversi, concatenato con altri flussi, seguito da dati estranei, e su un file a più frame ogni troncamento e un byte alterato in ogni posizione. Sui binari nativi controlla che il bytecode generato dal C coincida con quello del driver Python, che i file prodotti coincidano con il modello anche con altri preset, e le opzioni e i codici d'uscita di xz.

Sul testo klzma è alla pari con LZMA puro, che non ha né intestazione né CRC: 3483 contro 3485 byte sulla tesi, 3124 contro 3113 su Janus.c, 3340 contro 3321 sul README; batte xz su tutti e tre (3544, 3172, 3380). Rispetto a kgzip2 il guadagno è netto: kgzip2 produce il 9,6% di byte in più sulla tesi e il 60,3% in più sul testo ripetuto.

Dove il dizionario conta klzma fa meglio di tutti: il testo ripetuto scende a 1830 byte (LZMA 1899, xz 1956, bzip2 2182), il file periodico a 42 (LZMA 54). Il merito è dei conteggi: su dati stazionari un modello che non dimentica converge più in fretta dello scorrimento di LZMA. Sui byte casuali il blocco memorizzato tiene il file a 4112 byte, meno di xz (4156) e di bzip2 (4580). Sui file minuscoli pesano gli 8 byte di CRC (immagine 16×16: 56 contro 44). bzip2 resta il migliore sul testo normale da 8 KiB, dove la BWT ha un vantaggio sui blocchi piccoli.

Che cosa conviene di LZMA. Prima di portare in Kairos un pezzo di LZMA, lo si è misurato sul modello. Solo la parse lazy paga davvero (Tabella 3): dall'1,7 al 2,2% sul testo. Le distanze rep1–rep3 e il letterale "matched" aggiungono contesti che su 8 KiB restano poco allenati, e il risultato peggiora leggermente; lo stimatore KT aiuta sui dati regolari e peggiora sul testo.

Preset e dimensione dei blocchi. I blocchi sono indipendenti, e ognuno riparte da un dizionario vuoto: la dimensione dei blocchi conta molto più della profondità della ricerca. Sui sorgenti di Kairos (Tabella 4) si passa da 90015 byte con `-0` a 69297 con `-9`; a parità di blocco, da 32 a 64 candidati (`-6` contro `-7`) si guadagna lo 0,7%. Il tempo per byte invece quasi non dipende dal blocco: un blocco solo, con il binario, si comprime in 8 KiB in 5 s, 32 KiB in 23 s, 64 KiB in 46 s, 128 KiB in 83 s, 256 KiB in 162 s, cioè

variante	tesi8k.tex	janus8k.c	readme8k.md	doppio8k.te	cerchio64.p	periodico8k
golosa (prima versione)	3545	3183	3414	1856	122	43
+ lazy, b2 > best (scelta)	3483	3124	3340	1831	124	43
+ lazy, b2 > best + 1	3495	3140	3354	1833	127	43
+ stimatore KT (a = 1/2)	3549	3194	3424	1858	119	40
+ letterale matched	3552	3186	3420	1868	122	43
+ rep0–rep3	3553	3189	3415	1864	114	43
lazy + tutto il resto	3498	3141	3361	1852	112	41
LZMA (7-Zip, raw)	3485	3113	3321	1899	109	54

Tabella 3: Varianti misurate sul modello Python, con lo stesso rANS e gli stessi conteggi: i numeri sono i byte che produrrebbe la versione Kairos (intestazione stimata). La lazy con $b_2 > \ell$ è quella adottata. Lo stimatore KT è $p_0 = (n_0 + 1/2)/(n+1)$ invece di $(n_0+1)/(n+2)$.

preset	blocco (KiB)	candidati	byte	%
`-0`	8	4	90015	34,3%
`-1`	8	8	88879	33,9%
`-2`	16	8	83771	32,0%
`-3`	16	16	82902	31,6%
`-4`	32	16	78393	29,9%
`-5`	32	32	77711	29,6%
`-6`	64	32	74057	28,3%
`-7`	64	64	73515	28,0%
`-8`	128	64	71671	27,3%
`-9`	256	128	69297	26,4%
`-6e`	64	64	73515	28,0%
`-9e`	256	256	69089	26,4%
xz `-9e`			64124	24,5%
LZMA puro			64056	24,4%
bzip2 `-9`			64612	24,6%

Tabella 4: I preset su 256 KiB di sorgenti di Kairos (C, Python, Markdown), byte dal modello Python, che coincidono con quelli del binario. Il predefinito è `-6`.

circa in proporzione ai byte. Un blocco grande costa memoria (con 256 KiB la VM arriva a 92 MiB in decompressione, per ogni blocco in parallelo) e, sui file piccoli, parallelismo: per questo il predefinito è 64 KiB. Anche con `-9` klzma resta sopra xz `-9e` (69297 contro 64124 byte) e sopra LZMA puro, che vede tutto il file con un dizionario unico.

Dove va il tempo. La Tabella 5 misura ogni stadio con programmi troncati. La codifica rANS resta la parte più cara: ogni bit è una divisione, un aggiornamento del modello e, all'indietro, la stessa cosa al contrario. Nell'ultima versione la probabilità di ogni bit si calcola in linea in `codbit`, invece che con `call prob` e `uncall prob`; nel ciclo che misura una corrispondenza `if b[i+k] == b[j+k] then l += 1 fi` diventa `l += b[i+k] == b[j+k]`, perché un confronto vale 0 o 1; e le procedure con i cicli più percorsi stanno in testa al bytecode, perché la VM ritrova la riga di ogni salto scorrendo il testo dall'inizio (il rapporto su kgzip2 lo misura).

Efficienza. Sui file del corpus (Tabella 6) la compressione è in media 1,7 volte più veloce e la decompressione 1,6 volte, da 1,4 a 2,3 volte secondo il file.

Più blocchi e `par`. 20 KB della tesi in tre blocchi da 8 KiB, rami di un `par`: 8648 byte, compressione in 7,2 s di VM contro 13,4 s in sequenza (1,87×), decompressione in 7,7 s contro 14,3 s (1,85×), su 2 core. Lo stesso file come blocco unico: 7717 byte (bzip2 7154, xz 7528), in 13,9 s e 14,2 s. Con blocchi

stadio	v1	v2	ora	quota
catene di hash	0,05	0,05	0,25	2%
parse (con lazy)	3,41	3,33	2,55	23%
catene disfatte	-0,01	0,06	0,21	2%
istogramma del modello	9,89	3,16	3,43	31%
codifica (rANS)	11,13	8,18	4,71	42%
totale	24,47	14,78	11,15	100%

Tabella 5: Costo per stadio su un blocco di 8 KiB della tesi, andata e ritorno di ogni stadio (secondi di VM). v1: istogramma calcolato rieseguendo la codifica in un modo di conteggio e disfacendola; v2: conteggio in sola lettura, codifica senza i modi; ora: probabilità in linea, confronti come valori nella ricerca, procedure calde in testa al bytecode.

	prima (s)		ora (s)	
file	comprimi	`uncall`	comprimi	`uncall`
tesi (LaTeX)	7,5	7,7	5,5	5,8
Janus.c	6,6	6,8	4,6	5,0
README	6,8	7,1	4,8	5,2
testo ripetuto 2 volte	3,9	4,1	2,8	3,1
periodico	2,1	2,2	1,0	1,1
immagine 64×64	1,1	1,2	0,6	0,7
zeri	1,0	1,1	0,5	0,5
casuale	5,3	0,0	2,3	0,0

Tabella 6: Secondi per file prima e dopo le ottimizzazioni, stessa macchina e stessa VM. “Prima”: tempo della VM con il driver Python, prima degli interventi; “ora”: il comando nativo intero, dall’avvio alla fine. I file compressi sono identici.

indipendenti ogni blocco riparte da un dizionario vuoto: è il prezzo del parallelismo, lo stesso che paga xz in modalità multithread.

Streaming e memoria. Prima dello streaming l’involucro leggeva tutto il file e metteva tutti i blocchi in un solo programma: la memoria cresceva con il file, di circa 639 byte di RAM per byte d’ingresso, da 72 MiB per 16 KiB a 222 MiB per 256 KiB. Con lo streaming il picco smette di dipendere dal file: col driver Python stava fra 78 e 86 MiB, quasi tutti dell’interprete Python e del frontend; il binario nativo, che non li contiene, sta fra 11 e 52 MiB da 16 a 1024 KiB (Tabella 7), con blocchi otto volte più grandi. Il tempo invece resta lineare: 3,2 KiB al secondo in compressione e 3,0 in decompressione su 2 core. Un file da 100 GB si comprime con poche decine di MiB, ma richiederebbe circa 356 giorni: la memoria non è più il limite, il tempo sì.

Un file alterato. Il CRC di ogni frame ferma un file alterato prima della VM. Senza di esso (lo stesso esperimento del rapporto su kgzip2, `bench/manomissione.py`), su 254 alterazioni di un blocco di 1 KiB (bit invertiti, coppie di bit, contatori cambiati di uno) la VM si ferma da sola nel 98,4% dei casi e non gira mai all’infinito; il 1,6% termina con dati sbagliati, che il CRC dei dati ferma alla fine. Quasi sempre (231 casi) a fermarla è un indice fuori dai limiti: perso il sincronismo del rANS, presto una distanza punta prima dell’inizio del blocco. È lo stesso controllo che farebbe un decodificatore LZ scritto con cura; i controlli propri della reversibilità (`delocal`, asserzioni d’uscita) ne fermano solo 8. In kgzip2 la proporzione è opposta: lì sono loro a fermare il 83,1% delle alterazioni (il rapporto su kgzip2 descrive l’esperimento per intero).

	picco di memoria (MiB)			secondi (nativo)	
KiB	prima	streaming in Python	nativo	comprimi	`uncall`
16	72	78	11	11	11
64	104	81	30	47	47
256	222	81	52	87	93
1024	–	86	52	322	344

Tabella 7: Picco di memoria residente al crescere del file (sorgenti di Kairos concatenati). “Prima”: la versione che leggeva tutto il file e metteva tutti i blocchi in un solo programma (solo compressione, Python e VM insieme); “streaming in Python”: il driver Python a gruppi, blocchi da 8 KiB; “nativo”: il binario, con le opzioni predefinite (blocchi da 64 KiB, uno per core), massimo fra compressione e decompressione. 2 core.

6 Che cosa manca rispetto a LZMA

- **Parse ottima.** LZMA in modalità normale sceglie i gettoni minimizzando il costo stimato in bit; qui la parse è golosa con un passo di lazy matching, che basta già a portare klzma a ridosso di LZMA sul testo.
- **Ripetizioni.** LZMA ricorda le ultime quattro distanze (rep0–rep3) e ha la “short rep” di un byte; qui solo rep0 con lunghezza ≥ 2. Misurate sul modello, rep1–rep3 non pagano su blocchi da 8 KiB (Tabella 3).
- **Letterali dopo un match.** LZMA codifica il letterale che segue un match confrontandolo con il byte alla distanza rep0 (*matched literal*); qui il letterale ha sempre il solo contesto del byte precedente. Anche questa variante, misurata, non paga: più contesti si dividono gli stessi pochi dati.
- **Adattività.** I conteggi non dimenticano; lo scorrimento di LZMA segue meglio i dati non stazionari.
- **Dizionario.** Blocchi indipendenti da 8 a 256 KiB contro dizionari da megabyte.
- **Slot 12 e 13** con un bit diretto e l’align invece dell’albero inverso completo.

7 Limiti e lavori futuri

- **Velocità.** Poco più di un KiB al secondo per core: è un interprete di un linguaggio reversibile che rilegge ogni istruzione come testo, e la pulizia raddoppia parte del lavoro. Con lo streaming la memoria resta costante, quindi la dimensione dei file è limitata solo dal tempo.
- **Dizionario continuo fra blocchi**, come in LZMA2 vero, al posto dei blocchi indipendenti: rinuncia al `par` in cambio di rapporto.
- **Parse ottima reversibile**: i costi si possono calcolare dai conteggi, e la scelta di ogni gettone resterebbe una funzione pura disfatta subito, come ora.
- **Adattività**: uno stimatore che dimentichi, reversibile, per esempio conteggi dimezzati con il bit perso messo nel flusso compresso.

8 Conclusioni

Anche l’algoritmo di 7-Zip si scrive in Kairos una volta sola. Il codificatore entropico adattivo, che sembrava il pezzo più difficile da invertire, diventa pulito scegliendo il codificatore giusto (rANS invece del range coder) e l’ordine giusto (all’indietro, dall’istogramma). Sugli stessi file klzma comprime molto meglio di kgzip2, fa meglio di xz sul testo ripetuto e sui dati molto regolari, ed è alla pari con LZMA puro sul testo: la reversibilità, alla fine, non costa rapporto di compressione.

Riferimenti bibliografici

[1] J. Ziv, A. Lempel. *A Universal Algorithm for Sequential Data Compression*. IEEE Transactions on Information Theory 23(3), 1977.

[2] I. Pavlov. *LZMA SDK* e 7-Zip, `https://www.7-zip.org/sdk.html`.

[3] J. Duda. *Asymmetric Numeral Systems: Entropy Coding Combining Speed of Huffman Coding with Compression Rate of Arithmetic Coding*. arXiv:1311.2540, 2013.

[4] M. Lyngby, J. Nylandsted, R. Glück, T. Yokoyama. *Towards Clean Reversible Lossless Compression: A Reversible Programming Experiment with Zip*. RC 2024, LNCS 14680, pp. 94–102.

[5] C. H. Bennett. *Logical Reversibility of Computation*. IBM Journal of Research and Development 17(6), 1973.

[6] L. Collin et al. *XZ Utils*, `https://tukaani.org/xz/`.

A File e comandi

Tutto è in `klzma/` nella repository `kairos-compressori`: `klzma.kairos` (la libreria), `driver.py` (il riferimento Python: programma di un gruppo, ordine delle procedure), `modello.py` (la stessa pipeline in Python, con un decodificatore indipendente). Il comando è `nativo/klzma.c`, sopra il driver comune `nativo/kc.c`; la libreria in bytecode la produce `nativo/genera.py`. Il formato a frame e lo streaming di riferimento sono in `comune/contenitore.py` e `comune/flusso.py`, condivisi con kgzip2. In `bench/`: `misura.py`, `esperimenti_klzma.py` (le varianti della Tabella 3), `profilo_klzma.py`, `concorrenza.py`, `memoria.py`.

```
make eseguibili                        # dist/bin/klzma, dist/bin/kgzip2
klzma -v file.txt                      # file.txt.klz, poi rimuove file.txt
klzma -l file.txt.klz                  # flussi, blocchi, dimensioni, rapporto
klzma -d file.txt.klz                  # uncall dello stesso programma
klzma -9e -k file.txt                  # blocchi da 256 KiB, 256 candidati
tar c cartella | klzma > cartella.tar.klz    # streaming, memoria costante
cat a.klz b.klz | klzma -d > ab        # flussi concatenati
python3 bench/esperimenti_klzma.py test/corpus/*
```